

0811_김석우

1. Motion in-betweening

팩트체크 — "손 특화 motion in-betweening 사례가 적다"는 전제
Claude : "절반은 맞고, 절반은 정정이 필요합니다."

맞는 부분:

RMIB, Δ -Interpolator, TST, SILK, CondMDI, AIS 등 이 분야의 "메인 스트림" 논문들은 전부 LaFAN1·Human3.6M 같은 전신(全身) mocap 벤치마크에서만 학습·평가됩니다. 이 데이터들은 손을 아예 관절 하나로 뭉개거나(손목까지만), 손가락 관절을 따로 추적하지 않습니다. "손가락이 여러 개인 손 스켈레톤"을 대상으로 한 일반적인 in-betweening 연구는 확실히 거의 없습니다.

정정이 필요한 부분:

다른 이름으로 이미 존재하는 인접 분야가 있었습니다 — 수어(sign language) 생성 분야입니다.

- **SignSparK (2026)** — 수어 sparse keyframe → 전체 포즈 생성. 손을 MANO 15 관절+6D 회전으로 표현하고, SLERP 대비 오차를 약 50% 줄임. 코드도 공개 (GitHub). 우리가 하려는 것과 문제 정의가 사실상 동일합니다 — 단지 "애니메이션 제작 도구"가 아니라 "수어 문장 생성"이라는 다른 동기로 접근했을 뿐입니다.
- **Sign-D2C (CVPR 2025)** — 긴 수어 영상에서 구간을 랜덤 마스킹해서 학습, 확산모델로 빈 프레임 채움. CondMDI와 원리가 거의 같습니다.
- **GARD, RVQ-VAE** 보간 등도 같은 문제("gloss 사이 전이 프레임 생성")를 다룹니다.

결론:

"손 관절을 자세히 추적하는 in-betweening을 애니메이션 문제로서 다룬 일반 연구"는 정말 드문 게 맞다.

하지만, "손이 중심인 동작에서 빈 구간을 채우는 문제" 자체는 **수어 생성 쪽에서 이미 활발히 다뤄지고** 있었고, 그쪽 논문의 아이디어와 공개 코드를 그대로 참조할 수 있습니다. **오히려 이건 좋은 소식입니다** — 참고할 선행 구현체가 있다는 뜻이니깐요.

2. Dataset 선정

핵심 기준

1. 손가락 관절을 실제로 정확히 추적했는가 (= 뼈가 고정 길이인가)

▼ WHY ??

다수의 Motion in-betweening 논문들(RMIB, SILK, Δ -Interpolator 등)은 전부 "손목/어깨 같은 관절이 얼마나 회전했는가" 를 예측하고, 거기에 고정된 뼈 길이를 적용해서(forward kinematics, FK) 최종 위치를 계산하는 방식입니다.

이 방식이 성립하려면 전제가 하나 있어야 합니다 — 뼈 길이가 안 변한다는 것. 안 그러면:

- "회전"이라는 개념 자체가 애매해집니다. 같은 관절 위치라도 뼈 길이가 다르면 필요한 회전각이 달라지니, "정답 회전값"을 하나로 정의할 수가 없습니다.
- 그래서 이 분야 표준 평가지표인 L2Q(회전 오차), NPSS(회전의 주파수 패턴) 자체를 계산할 방법이 없어집니다.
- 지난번 SHREC/DHG로 할 때 실제로 이 문제에 부딪혔습니다 — 뼈 길이가 프레임마다 최대 20~37% 흔들려서, 회전 기반 표현을 포기하고 위치 기반의 독자적인 방식을 새로 설계해야 했습니다. 그러면 기존 논문의 아키텍처·손실함수·평가지표를 그대로 못 쓰고, 우리가 직접 만든 방식이 맞는지 우리도 다 검증해야 했습니다 (그때 회�행렬 직교성, SE(3) 불변성 같은 걸 일일이 검증한 이유가 이겁니다).

즉, 뼈가 고정된 데이터를 쓰면 이 분야에서 가장 많이 검증된 표준 레시피를 그대로 가져다 쓸 수 있어서, 구현 리스크가 확 줄고 발표할 때도 "기존 논문 수치와 직접 비교 가능하다"는 설득력이 생깁니다.

2. 물체 조작이 아니라 손 자체의 다양한 움직임인가

▼ WHY ??

이건 데이터의 동작 분포가 우리가 풀려는 문제와 맞는가의 문제입니다.

GRAB, ARCTIC, OakInk 같은 데이터셋은 대부분 "컵을 잡는다", "리모컨을 돌린다" 같은 물체 조작이 중심입니다. 이런 데이터로 학습하면:

1. 손 모양이 물체 형태에 종속됩니다. 손가락이 어떻게 구부러지는지가 "그 사람이 어떻게 움직이고 싶은가"가 아니라 "잡고 있는 물체가 어떤 모양인

가"로 대부분 결정됩니다. 그러면 모델이 배우는 건 "손의 움직임 자체의 패턴"이 아니라 "이 물체를 쥐려면 이 모양이어야 한다"는, 우리가 원하는 것과 다른 종류의 지식이 됩니다.

2. 동작의 다양성이 좁습니다. 대부분의 시간이 "물체로 다가가서 → 쥐는 자세로 고정"이라, 손가락이 리드미컬하게 계속 움직이는 구간(수어, 제스처, 흔들기 같은)이 적습니다. In-betweening이 진짜 어려워지는 지점은 손이 계속 활발하게 모양을 바꾸는 구간인데, 물체 조작 데이터엔 그런 구간이 상대적으로 부족합니다.
3. 애초에 이건 다른 연구 분야입니다. "손-물체 상호작용 생성"은 이미 ManiDext, GraspXL 같은 이름으로 따로 연구되고 있는 별개 주제입니다. 이걸 섞으면 우리가 하려는 "손 자체의 동작 in-betweening"이라는 논지가 흐려지고, 물체 정보(위치·형태)까지 모델에 같이 넣어줘야 하는 훨씬 큰 문제로 커집니다.

3. 지금 당장 받을 수 있는가?

▼ WHY ??

당장 다운 못 받으면 답답하니까 ππππ

후보	확인 결과	채택 여부
SHREC'17 + DHG	기존에 보유하던 손 제스처 데이터. 카메라 1대짜리 SDK 추정이라 손가락 뼈 길이가 프레임마다 최대 20% 흔들림(강제 아님) → 표준 회전 기반 표현을 못 씀. 시퀀스도 짧아(중앙값 54프레임) 이 분야 표준 벤치마크 길이(T=45)를 못 씀	❌ 배제
GRAB / InterHand2.6M / OakInk2 / DexYCB	문헌 조사 결과 대부분 "물체를 잡고 조작하는" 동작 중심. GRAB은 실제 신청·승인 절차가 필요해 접근 마찰도 있음	❌ 배제
GigaHands (CVPR 2025)	실제로 전체 주석 1만3787개를 다운받아 텍스트 분석함 → 잡다/쥐다/놓다 같은 물체조작 동사가 36.6%, 흔들다/가리키다 같은 순수 제스처 동사는 2.1%뿐. 즉 사실상 전부 물체 조작 데이터였음 (뼈 길이 자체는 안정적이었음에도 동작 성격이 안 맞음)	❌ 실측 후 배제
SignAvatars (수어 데이터)	관절 위치가 실측(mocap)이 아니라 단안 영상에서 AI로 추정한 값이라 노이즈 위험	❌ 보류
CSL-Daily + How2Sign (SignSpark)	손을 회전 파라미터(MANO 계열)로 저장해 뼈 길이 문제가 원천적으로 없음 , 물체 없이 손 모양만 계속	✅ 최종 채택

후보	확인 결과	채택 여부
전처리본)	바뀌는 동작, 등록 절차 없이 바로 다운로드 가능. 실제로 train+dev 5만 클립 전체를 스캔해 검증: 시퀀스 길이 중앙값 113~137프레임(SHREC의 2~3배), T=45 커버리지 87~94%, 손 회전이 실제로 계속 유의미하게 변함, 한손만 쓰는 클립도 1.6~3.2% 뿐	

3. Model Idea

3.1. 참조한 기존 연구·모델 Idea

3.1.1. 몸통 in-betweening 계보 (구조·손실 설계 참고용)

- RMIB(2020) — "목표까지 남은 시간"을 모델에 알려주는 time-to-arrival 임베딩
- Δ -Interpolator(2022) — 절대 위치 대신 보간 결과 대비 잔차를 예측, 마지막 관측 프레임 기준 로컬 좌표계 사용
- TST(2022) — 큰 그림(궤적)과 디테일(관절 모양)을 2단계로 나눠 생성
- **SILK(2025)** — 복잡한 구조 없이 트랜스포머 인코더 하나 + 표현 설계만으로 최고 성능. L1 손실 하나, 속도는 입력에만 사용. **가장 단순하고 검증된 레시피라 1순위 참고**

▼ 자세히

아키텍처

- 트랜스포머 인코더 1개, 6층, 8헤드, 은닉차원(d_model) 1024, FFN 차원 4096
- 위치 인코딩: 절대 위치가 아니라 "프레임 간 거리" 기반의 학습된 상대 위치 인코딩 → 그래서 학습 때 안 본 길이도 추론 때 처리 가능
- 마스킹 없이 모든 프레임이 서로를 다 봄 (context든 빈칸이든 target이든 어텐션 대상 동일)

입출력 특징 구성 (관절 수 J개 기준)

	차원	구성
입력	18J+8	루트 위치(2)+루트 방향(2)+루트 선속도(2)+루트 각속도(2) + 관절위치(3J)+관절방향(6J)+관절 선속도(3J)+관절 각속도(6J)
출력	9J+4	루트 위치(2)+루트 방향(2) + 관절위치(3J)+관절방향(6J) — 속도는 출력에서 빠짐

빈 구간(빈칸) 프레임은 **0으로 채움** (선형보간으로 미리 채우지 않음).

학습 설정

- 손실: 전 특징에 대해 **L1 하나만** (위치/회전/속도 따로 가중치 안 줌)
- 옵티마이저: AdamW + noam 러닝레이트 스케줄, 배치 64
- 데이터: LaFAN1(약 4.5시간 분량)
- 윈도우 샘플링 간격을 **20 → 5로 촘촘하게** 해서 학습 데이터를 4배로 늘림
- 학습 구성: context 10 + 빈칸 5~30프레임(매번 랜덤) + target 1 → **한 모델이 여러 빈칸 길이를 한 번에 학습**
- 학습 때 최대 30프레임까지만 봤는데, 추론 때 **45프레임에도 재학습 없이 잘 일반화됨**

Ablation(무엇이 진짜 효과 있었나) — 전 부 T=30, L2P 기준

바꾼 것	결과
전역/국소 분리 표현(root space) vs 단순 로컬 표현	0.83 vs 1.00 (17% 개선)
속도를 입력에만 넣기 vs 속도 없이 위치만	0.83 vs 1.00 (17% 개선)
빈칸을 0으로 채움 vs SLERP로 미리 채움	0.83 vs 0.91 (9% 개선)
샘플링 간격 5 vs 20 (데이터 양)	0.83 vs 0.98 (15% 개선)

결론 문장 그대로: "모델 복잡도보다 데이터를 어떻게 다루느냐가 성능을 좌우한다." 구조는 트랜스포머 인코더 하나로 충분하고, 표현 설계·데이터 양·전처리 방식이 진짜 승부처였다는 게 이 논문의 핵심 주장입니다.

3.1.2. 수어 생성 분야 (손 특화 아이디어 원천)

- **SignSparK(ECCV 2026)** — 드문드문 주어진 키프레임으로 전체 수어 동작을 생성. Conditional Flow Matching이라는 최신 생성 기법 사용, 손을 회전 파라미터로 표현,

SLERP 대비 오차 약 50% 감소. 코드와 전처리 데이터가 공개돼 있어 그대로 가져다 쓸 수 있음

▼ 자세히

파이프라인이 **2단계**로 구성됩니다: ① 자연 키프레임을 자동으로 찾는 FAST, ② 그 키프레임으로 전체 동작을 채우는 SignSpark 본체.

① FAST — 자연 키프레임(수어 경계) 탐지

- 양손 각각을 독립적인 시공간 스트림으로 처리(Two-Stream Hand Encoder): 1D-Conv로 시간축 처리 + 스트림을 섞는 Mixer + 트랜스포머
- 프레임마다 **BIO 태그**(B=수어 시작/I=수어 진행중/O=수어 아님)를 예측 → 이게 바로 저희가 지난번 CSL-Daily/How2Sign 데이터에서 직접 확인했던 **segment 필드(0/1/2)** 그 자체입니다. 저희가 EDA로 "수어당 평균 17~18프레임" 이라고 잔 게 정확히 이 FAST 모델의 출력을 쓴 것이었습니다.
- 기존 스켈레톤 기반 구간 탐지 방법 대비 **45배 빠르고 32배 가벼움**

② SignSpark 본체 — Conditional Flow Matching

- **Flow Matching**: 확산모델(diffusion)과 같은 계열의 생성 기법인데, "노이즈 → 정답"으로 가는 경로를 직접 벡터장(velocity field)으로 학습합니다. 확산모델은 보통 수십~수백 스텝을 거쳐야 하는데, 여기서 **"reconstruction regularization"** 덕분에 **10스텝 미만**으로 샘플링 가능 — 훨씬 빠릅니다.
- **조건**: 각 수어의 시작·중간·끝 프레임(onset → midpoint → offset, FAST가 찾아준 것) + 발화 텍스트(Multilingual-CLIP 임베딩) + timestep
- **Classifier-Free Guidance** 사용 — 조건을 얼마나 강하게 따를지 추론 때 조절 가능 (텍스트 조건을 빼고 순수 키프레임만으로도 돌릴 수 있다는 뜻이라, 저희가 텍스트/gloss 조건을 걷어내려는 계획과 구조적으로 잘 맞습니다)
- **손 표현**: MANO 15관절 × 6D 회전 — 저희가 실제로 다운받아 확인한 CSL-Daily/How2Sign 데이터 포맷과 **정확히 일치**합니다
- **스트림별 독립 학습**: 손/몸통/얼굴을 따로 학습한 뒤 합침. 저희는 손 스트림만 필요

- 학습 규모(공개 코드 기준): 배치 128, 50만 스텝. Phoenix14T, CSL-Daily, How2Sign + BOBSL 10%로 학습

결과 (Keyframe-to-Pose 과제, DTW-J 점수, 낮을 수록 좋음)

	SLERP	SignSpark
몸통 기준 (3개 데이터셋)	3.71~4.36	1.68~2.80

데이터셋에 따라 오차를 40~55% 줄임.

- Sign-D2C(CVPR 2025) — 긴 영상 구간을 랜덤으로 지워서 학습, 확산모델로 채우기 (CondMDI와 유사한 원리)

3.2. 그래서 뭐 선택해요 ?

트랙 A — 검증된 표준 레시피를 손 스케레톤에 적용

- **SILK 논문**의 방식(단일 트랜스포머 인코더, 손목 위치/회전과 손가락 회전을 나눠 표현, L1 손실)을 그대로 가져와 손 데이터에 맞게 축소 적용
- 몸통용으로 설계된 이 방식의 요소들(속도 입력, 상대 위치 인코딩 등)이 관절이 훨씬 작고 촘촘하고 빠르게 움직이는 손에도 그대로 유효한지 검증하는 것이 핵심 관전 포인트
- 뼈가 고정 길이인 데이터라 기존 논문의 표준 평가지표(L2Q, NPSS)를 그대로 쓸 수 있어 **발표된 논문 수치와 직접 비교 가능**

트랙 A 정리

	트랙 A
가져오는 것	SILK의 단순 트랜스포머 인코더 + 표현 설계 (전역/국소 분리, 속도는 입력에만, 0으로 채움)
걸어내는 것	(원 논문이 이미 몸통용이라 관절 수만 줄이면 됨)
리스크	낮음 — 구조가 단순해서 구현 부담 적음

트랙 B — 인접 분야(수어 생성)의 아이디어를 일반화

- SignSpark의 실제 아키텍처(Conditional Flow Matching + 키프레임 조건화)를 가져오되, 수어 특화 조건(텍스트, gloss)은 빼고 순수하게 "context+target → transition" 문제로 재구성

- 공개된 코드를 스캐폴드로 활용할 수 있어 처음부터 새로 만드는 것보다 리스크가 낮음
- "손 동작 자체를 위한 일반적 도구로 재구성했다"는 서사가 뚜렷해 발표 임팩트가 큼

트랙 B 정리

	트랙 B
가져오는 것	SignSpark의 Conditional Flow Matching + 키프레임 조건화
꺼내내는 것	텍스트/gloss 조건, 다국어 처리 부분
리스크	더 낮음 — 데이터 포맷이 이미 SignSpark가 쓰던 것과 100% 동일해서, 공개 코드에서 텍스트 조건 분기만 빼면 거의 그대로 돌아갈 가능성이 큼

요약하면: 두 트랙은 "같은 데이터로, 검증된 단순한 방법(A)과 인접 분야의 특화된 방법(B)을 나란히 비교"하는 구도입니다.

4. 진행 상황, 앞으로의 방향 정리

4.1. 지금까지 진행 상황

1단계: 주제 전환 + SHREC/DHG 인프라 구축 — 완료했다가 방향 전환됨

처음엔 SHREC'17/DHG로 in-betweening 파이프라인을 끝까지 만들었습니다 (좌표계 설계, EDA, PyTorch Dataset, 베이스라인, 논문 조사, 벤치마크 프로토콜 문서까지 — 코드/문서 10개 이상). 그런데 "데이터 개수를 단순 합산해도 되냐"는 지적을 받고 검증해보니, 물체 조작 데이터든 뭐든 성격이 다른 걸 억지로 합친 것들이 있었고, 애초에 SHREC/DHG 자체도 손 특화 in-betweening 문헌이 거의 없다는 걸 알게 되면서 데이터 소스 자체를 다시 검토하게 됐습니다.

→ 이 단계 산출물은 남아있지만, 지금은 우선순위가 낮아진 상태입니다 (완전 폐기는 아니고, 나중에 cross-check용으로 쓸 수도 있음).

2단계: 데이터 재탐색 — 완료

- "손 특화 in-betweening이 정말 드문지" 팩트체크 → 몸통 연구엔 없지만 **수어 생성 분야에 인접 연구가 있다는 걸 발견**
- GigaHands 후보 검토 → 실제로 다운받아 주석 1만3787개 다 읽어보고 **물체조작이 압도적이라 배제**

- CSL-Daily + How2Sign(SignSpark 전처리본) 발견 → **train+dev 전체 (약 15GB, 5만 클립) 실제로 다운로드하고 전수 스캔 EDA 완료** (시퀀스 길이, 손 회전 움직임, 윈도우 확보량, 한손우세 비율 등 전부 실측)

→ **여기까지가 "지금 확정된 것"입니다.** 데이터는 CSL-Daily+How2Sign으로, 트랙 A(SILK 스타일 단순 트랜스포머)/트랙 B(SignSpark 아키텍처 응용) 두 갈래로 간다는 기획까지 정리됐습니다.

4.2. 아직 안 한 것 (다음 단계)

항목	상태
새 데이터용 좌표/특징 표현 모듈 (<code>ib_core.py</code> 에 해당하는 것)	✗ SHREC용으로 만든 건 위치 기반이라 안 맞음. 회전 기반으로 새로 설계 필요
PyTorch Dataset (<code>ib_dataset.py</code> 에 해당하는 것)	✗ LMDB에서 context+transition+target 윈도우를 뽑는 로더 아직 없음
벤치마크 프로토콜 재정의	✗ context/transition 길이는 EDA로 후보(T=45까지 가능)만 확인했고, 공식으로 확정은 안 함. train/val/ test 분할도 test셋 자체를 아직 안 받음
CSL-Daily(96차원) vs How2Sign(90차원) 관절 수 불일치 처리	✗ 결정 안 됨 (공통 15관절만 쓰지 등)
베이스라인(SLERP 등) 수치	✗ SHREC 때는 냈지만 새 데이터로는 아직 안 냄
실제 모델 코드(트랙 A/B)	✗ 설계만 하고 코드 없음
학습	✗ 당연히 아직